**ARRAY IMPLEMENTATION OF QUEUE:**

**CODE & OUTPUT-**

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 50

int queue[MAX],n,front=-1,rear=-1;

void qinsert(){
    int item;
   printf("\nEnter the element\n");
   scanf("\n%d",&item);
   if(rear == MAX-1)
   {
      printf("\nOVERFLOW\n");
      return;
   }
   if(front == -1 && rear == -1)
   {
      front = 0;
      rear = 0;
   }
   else
   {
      rear = rear+1;
   }
   queue[rear] = item;
   printf("\nValue inserted ");
```

```
}

void qdelete(){
   int item;
   if (front == -1 || front > rear)
   {
      printf("\nUNDERFLOW\n");
      return;

   }
   else
   {
      item = queue[front];
      if(front == rear)
      {
         front = -1;
         rear = -1 ;
      }
      else
      {
         front = front + 1;
      }
      printf("\nvalue deleted ");
   }

}

void qdisplay(){
```

```
    int i;
    if(rear == -1)
    {
        printf("\nEmpty queue\n");
    }
    else
    {   printf("\nprinting values .....\n");
        for(i=front;i<=rear;i++)
        {
            printf("\n%d\n",queue[i]);
        }
    }
}

void main(){
    int choice;
    do{
        printf("\n1.insert an element\n2.Delete an element\n3.Display the queue\n4.Exit\n");
        printf("\nEnter your choice ?");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1:
            qinsert();
            break;
            case 2:
            qdelete();
            break;
            case 3:
```

```
            qdisplay();
            break;
            case 4:
            exit(0);
            break;
            default:
            printf("\nEnter valid choice??\n");
        }
    }while(choice!=4);
}
```

```
1.insert an element
2.Delete an element
3.Display the queue
4.Exit

Enter your choice ?1

Enter the element
12

Value inserted
1.insert an element
2.Delete an element
3.Display the queue
4.Exit

Enter your choice ?1

Enter the element
24

Value inserted
1.insert an element
2.Delete an element
3.Display the queue
4.Exit
```

```
Enter your choice ?1

Enter the element
36

Value inserted
1.insert an element
2.Delete an element
3.Display the queue
4.Exit

Enter your choice ?3

printing values .....

12

24

36

1.insert an element
2.Delete an element
3.Display the queue
4.Exit
```

```
Enter your choice ?2

value deleted
1.insert an element
2.Delete an element
3.Display the queue
4.Exit

Enter your choice ?3

printing values .....

24

36

1.insert an element
2.Delete an element
3.Display the queue
4.Exit

Enter your choice ?4


=== Code Execution Successful ===
```

**LINKED LIST IMPLEMENTATION OF QUEUE:**

**CODE & OUTPUT-**

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

typedef struct {
    Node* front;
    Node* rear;
} Queue;

void initializeQueue(Queue* q) {
    q->front = NULL;
    q->rear = NULL;
}

// Check if the queue is empty
int isEmpty(Queue* q) {
    return q->front == NULL;
}

void enqueue(Queue* q, int value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
```

```
    if (!newNode) {
        printf("Memory allocation failed\n");
        return;
    }
    newNode->data = value;
    newNode->next = NULL;
    if (isEmpty(q)) {
        q->front = newNode;
    } else {
        q->rear->next = newNode;
    }
    q->rear = newNode;
    printf("Enqueued: %d\n", value);
}

int dequeue(Queue* q) {
    if (isEmpty(q)) {
        printf("Queue is empty\n");
        return -1;
    }
    int item = q->front->data;
    Node* temp = q->front;
    q->front = q->front->next;
    if (q->front == NULL) {
        q->rear = NULL;
    }
    free(temp);
    printf("Dequeued: %d\n", item);
    return item;
```

```
}

void displayQueue(Queue* q) {
    if (isEmpty(q)) {
        printf("Queue is empty\n");
        return;
    }
    Node* temp = q->front;
    printf("Queue: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    Queue q;
    initializeQueue(&q);

    int choice, val;
    do{
        printf("\n1.insert an element\n2.Delete an element\n3.Display the queue\n4.Exit\n");
        printf("\nEnter your choice ?");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1:
                printf("Enter the value to enqueue: ");
```

```
            scanf("%d", &val);
            enqueue(&q,val);
            break;
        case 2:
            dequeue(&q);
            break;
        case 3:
            displayQueue(&q);
            break;
        case 4:
            exit(0);
            break;
        default:
            printf("\nEnter valid choice??\n");
      }
   }while(choice!=4);
}
```

```
1.insert an element
2.Delete an element
3.Display the queue
4.Exit

Enter your choice ?1
Enter the value to enqueue: 11
Enqueued: 11

1.insert an element
2.Delete an element
3.Display the queue
4.Exit

Enter your choice ?13

Enter valid choice??

1.insert an element
2.Delete an element
3.Display the queue
4.Exit

Enter your choice ?1
Enter the value to enqueue: 33
Enqueued: 33
```

```
1.insert an element
2.Delete an element
3.Display the queue
4.Exit

Enter your choice ?1
Enter the value to enqueue: 55
Enqueued: 55

1.insert an element
2.Delete an element
3.Display the queue
4.Exit

Enter your choice ?3
Queue: 11 33 55

1.insert an element
2.Delete an element
3.Display the queue
4.Exit

Enter your choice ?2
Dequeued: 11
```

```
1.insert an element
2.Delete an element
3.Display the queue
4.Exit

Enter your choice ?3
Queue: 33 55

1.insert an element
2.Delete an element
3.Display the queue
4.Exit

Enter your choice ?4


=== Code Execution Successful ===
```